

Loop vs Graf – cue cards

12 min · svenska · samma numrering som MANUS.md

Kort 0 · Innan du går upp

- Terminal A + B i repot, `claude` startat i båda. `npm run reset` körd. Ralph-raden i klippbordet.
- Chrome: artifact, scen 1, `F`. Presenter-panel av.
- Första gången `claude` startas i repot: trust-dialogen har "No, exit" förvalt → pil ner + Enter.
- Inget nät? Kör bara artifacten + `demo/runs/`.

```
/ralph-loop:ralph-loop Triagera alla tickets enligt TRIAGE.md --completion-promise "ALLA TICKETS TRIAGERADE" --max-iterations 8
```

Tangenter: ←/→ scen · Space spela · R om · N stödord · F fullskärm · E lägg till kant (scen 4)

1 • Hook

0:00 • Chrome scen 1, stilla

- Två sätt att bygga en agent för samma uppgift: fem supportärenden.
- Loop engineering. Graph engineering.
- Frågan är inte vilket som är bäst. Frågan är: vad vet du i förväg?

2 · Förklara loopen

1:00 · Space

- Prompt + stoppvillkor. Tänk, agera, observera, klar? Ett varv till.
- Vägen väljer modellen. Jag äger målet.
- Claude Code är redan en loop. Ralph = samma prompt igen tills promise-frasen är sann.
- Stoppvillkoret är `npm run check` . Grönt = klart.

3 · Starta loopen

2:00 · Terminal A

- **GÖR: klistra in Ralph-raden, Enter (ev. två). Tillbaka till Chrome.**
- En rad. Uppgift i fil, promise som stopp, max åtta varv som säkerhetsbälte.
- "Nu jobbar den. Vi låter den vara."

4 • Förklara grafen

3:00 • Chrome scen 2, Space

- Jag ritar noderna i förväg: Inbox → Klassificera → Kund || Policy → villkor → Svara | Eskalera → Verifiera.
- Varje nod: bundet anrop med schema. Kanterna är kod.
- Gratis: parallellism, observerbarhet, känd kostnad. Priset: du måste veta strukturen innan.

5 · Starta grafen

4:00 · Terminal B

- **GÖR:** `/trriage-graph` , Enter (ev. två). Peka på faserna. Tillbaka till Chrome.
- Claude Codes Workflow. Faserna = grafen på riktigt. 22 anrop, alla med schema.

6 · Sida vid sida

5:00 · Chrome scen 3, Space

- Fyra vanliga ärenden. Båda gröna.
- Loopen: elva steg, okänt i förväg. Grafen: tjugo, exakt som ritat.
- Loopens kostnad okänd tills klar. Grafens bunden innan start.

7 • Edge case

6:30 • Chrome scen 4, Space • vänta på rött • fråga • E

- Ärende fem: grekiska, GDPR, ingen kategori passar.
- Loopen: check säger nej, ett varv till, grönt. Improviserade.
- Grafen: "other", ingen kant. Stopp.
- **Fråga rummet: "Vad gör man med grafen nu?" Paus. Tryck E.**
- Kanten du lägger till efter att den bitit dig.
- Grafen kräver att du såg det komma. Loopen kräver att du litar på modellen.

8 • Terminalerna

8:00 • B, sen A, sen `npm run diff`

- B: "Grafen är klar. Grönt. 22 anrop, noll fel."
- A, ett varv: "Det visste jag inte i förväg. Poängen med ett stoppvillkor: jag behöver inte veta."
- A, flera varv: "Röda rader som blir färre. Det är loopen."
- diff: "Samma beslut. Två gröna. Två olika spår. Loopens läser jag i efterhand, grafens ritade jag i förväg."

9 • Hybrid

10:00 • Chrome scen 5, Space

- Zooma in i en nod. Där sitter en loop.
- Varje agentanrop i Workflow = en Claude Code-loop med eget stoppvillkor.
- Struktur utanpå, frihet inuti. Inte loop eller graf: graf med loopar i noderna.

10 • När använda vad

11:00 • Chrome scen 6, Space

- Loop när du inte vet vägen: utforskande, "fixa tills grönt", prototyper.
- Graf när du vet vägen och den ska köras hundra gånger: produktion, revision, kostnad.
- Oftast båda: graf för det du vet, loop i noderna för det du inte vet.
- **Fråga: var har ni loopar som borde vara grafer, och grafer som borde vara loopar?**

11 • Klart

12:00

- Q&A. github.com/zabirtech/loop-vs-graph